Use of AI-driven Code Generation Models in Teaching and Learning Programming: a Systematic Literature Review

Automated LLM-assisted Review Pipeline

Abstract

This systematic literature review examines the use of LLM-based code generation models in programming education. From 6,291 downloaded records, a top-ranked screening pool of 1,000 was assessed, yielding 159 included studies after evidence-card completeness and extraction audit. The analysis identified eight themes—Pedagogical Practices and Curriculum Integration, Model Capabilities and Performance Evaluation, Impact on Student Learning and Behavior, Instructor Perspectives and Adaptation, Academic Integrity and Plagiarism Detection, Automated Feedback and Learning Scaffolding, Student-Tool Interaction and Prompting Strategies, and Ethical, Societal, and Future Considerations—with studies potentially contributing to multiple themes. Across themes, a cross-cutting pattern emerges: AI code generators excel at routine tasks and feedback generation yet struggle with complex, security-critical, or domain-specific problems. Pedagogical innovations, such as Prompt Problems, reframe programming exercises around prompt engineering, while evaluation trends shift from accuracy metrics toward broader measures of pedagogical quality and student interaction. Instructors express divergent views on banning versus integrating these tools, and the unreliability of AI-generated text detectors underscores persistent academic integrity challenges. Key evidence gaps include the lack of longitudinal studies on learning outcomes, unclear guidelines for curriculum redesign, and insufficient strategies to mitigate over-reliance. The findings highlight both the transformative potential and the critical need for careful, evidence-based adoption of AI code generation in programming education.

1 Introduction

1.1 Background

The rapid advancement of large language models (LLMs) such as GPT-3, GPT-4, Codex, and their integration into tools like GitHub Copilot and ChatGPT has opened new possibilities for programming education. These models can generate code from natural language descriptions, offer debug suggestions, and provide explanations, thereby acting as interactive assistants for both learners and instructors. The potential benefits include reduced time on routine coding tasks, on-demand scaffolding, and exposure to diverse coding styles. However, the same capabilities raise concerns about academic integrity, over-reliance on automation, and the erosion of foundational programming skills. As these models become more accessible, educators and researchers are exploring how to harness them effectively while mitigating risks. This emerging area sits at the intersection of computing education, human-computer interaction, and artificial intelligence, prompting a need for a consolidated understanding of existing empirical work.

1.2 Motivation and gap

Despite a surge in publications examining the use of AI-driven code generation models in teaching and learning programming, the literature remains fragmented. Individual studies often focus on

specific tools, course levels, or evaluation metrics, making it difficult to discern broader patterns, common practices, and persistent challenges. No prior systematic literature review has synthesized the available evidence across the full spectrum of pedagogical contexts, model characteristics, and assessment approaches. In particular, there is no aggregated view of how these models are being integrated into curricula, what technical and pedagogical considerations arise, and how their effectiveness is measured. Filling this gap is essential to provide a foundation for future research, inform instructional design, and guide the responsible adoption of generative AI in programming education.

1.3 Review objective

To address this gap, we conducted a systematic literature review of 21 papers published since 2018, focusing on the use of LLM-based code generation models to support the teaching and learning of programming. The objective is to pinpoint the current state of research by synthesizing the reported practices, model characteristics, evaluation indicators, and the key aspects that practitioners and researchers should consider when integrating these models into educational settings.

1.4 Research questions

The review is guided by the following research questions:

1. What are the practices that use code generation models for teaching and learning programming?
2. What are the characteristics of the code generation models that are used in teaching and learning practices?
3. What indicators are used for evaluating the performance of code generation models in teaching and learning practices?
4. What aspects should be considered when utilizing code generation models in teaching and learning practices?

1.5 Contributions

This review contributes (a) a mapping of existing practices that employ code generation models in programming education, structured by pedagogical context and usage scenario; (b) a characterization of the models and their configurations used in these practices; (c) an overview of the evaluation indicators applied to assess model performance and educational outcomes; and (d) a synthesis of the considerations—technical, pedagogical, ethical, and practical—that emerge when integrating such models into teaching and learning. By consolidating findings across 21 studies, we provide a resource for educators, tool developers, and researchers aiming to navigate the opportunities and challenges of AI-driven code generation in programming education.

2 Methods

2.1 Review design and objective

We conducted a systematic literature review to examine the use of LLM-based code generation models in teaching and learning programming. The review was guided by four research questions: what practices use such models, what characteristics the models have, what indicators evaluate their

performance, and what aspects should be considered when deploying them. The review includes 159 papers published between 2018 and 2023.

2.2 Search sources and keyword strategy

Ten bibliographic databases were searched: ACM Digital Library, Scopus, Google Scholar, OpenAlex, Crossref, Semantic Scholar, Europe PMC, PubMed, DBLP, and IEEE Xplore. The search combined terms related to AI-driven code generation (e.g., "code generation model", "LLM", "ChatGPT", "Codex", "GitHub Copilot", "GPT-3", "GPT-4", "AI coding assistant", "AI code-generators") with educational terms (e.g., "teaching", "learning", "programming education", "computer science education", "curriculum", "assessment") and methodology terms (e.g., "empirical study", "experiment", "evaluation", "performance", "accuracy"). Boolean operators and wildcards were adjusted per database syntax. The initial search retrieved 6,291 records.

2.3 Eligibility criteria

Studies were included if they were (1) journal articles or conference proceedings written in English, (2) published between 2018 and 2023, (3) presented or discussed the use of LLM-based code generation models for educational purposes, and (4) reported empirical data (e.g., experiments, evaluations, methodology, results). Exclusion criteria eliminated papers not written in English, not about programming education, not using large language models, lacking empirical data, or being inaccessible (e.g., opinion pieces, overviews without original data). Reviews were excluded to avoid double counting of primary studies.

2.4 Article retrieval, cleaning, and ranking

After removing duplicates and cleaning records using DOI metadata, 2,471 unique records remained. Each record's title and abstract were ranked by relevance using a scoring strategy that prioritised terms from the research questions and eligibility criteria (e.g., "teaching", "learning", "code generation", "LLM", "empirical"). The top 1,000 ranked records formed the screening pool. From this pool, the 200 highest-ranked records were targeted for full-text screening and evidence extraction.

2.5 Full-text acquisition and LLM-assisted evidence-card extraction

Full-texts were retrieved for 199 of the 200 targeted records; one record was unavailable and was represented only by its abstract and metadata. For each record, an evidence card was generated using a large language model (LLM) that extracted structured fields (e.g., model used, educational context, evaluation indicators, findings). All 200 records were processed as single chunks. After independent LLM-assisted screening and a human decision audit, 159 records were classified as "include", 35 as "exclude", and 6 as "uncertain". The six uncertain records were resolved by re-examining the evidence cards and full-texts, leading to a final set of 159 studies for synthesis.

2.6 Evidence-card completeness and synthesis preparation

Before synthesis, each evidence card underwent an evidence-card completeness and extraction audit to verify that key fields (study design, model details, educational setting, outcome measures, limitations) were consistently and accurately captured. No formal quality scores were assigned; the audit focused on completeness and internal consistency. Incomplete cards were supplemented

by re-consulting the full-text. The 159 audited cards formed the basis for thematic analysis and synthesis across the included studies.

3 Results

3.1 Study selection flow

The database search yielded 6,291 records. After cleaning and deduplication based on DOI metadata, 2,471 records remained. These records were ranked by relevance using abstract and title screening, and the top 1,000 records formed the screening pool. From this pool, the top 200 records were selected for full-text retrieval and evidence-card generation by the LLM. Of the 200 evidence cards produced, 159 met the inclusion criteria, 35 were excluded, and 6 were marked as uncertain. The 159 included studies constituted the corpus for synthesis. The selection process is summarized in Figure 1.

3.2 Candidate metadata and ranking coverage

All 2,471 records with available DOI metadata were ranked based on their abstract and title relevance scores. The top 1,000 ranked records were designated as the screening pool, within which the most relevant 200 were targeted for detailed LLM-based screening. This two-stage ranking ensured that the most relevant studies were prioritized for evidence extraction.

3.3 Full-text availability

Full-text documents were successfully retrieved for 199 of the 200 article cards. One card relied solely on abstract metadata because no full text could be obtained. All 200 cards were processed as single chunks, with no card requiring segmentation beyond one chunk.

3.4 LLM screening decisions

The LLM screening classified 159 articles as included, 35 as excluded, and 6 as uncertain. The 159 included articles formed the basis for the synthesis. The 6 uncertain articles were not used in the final analysis, as they lacked clear decision evidence.

3.5 Evidence-card completeness and extraction audit

An evidence-card completeness and extraction audit was conducted to verify that all 200 article cards contained the required metadata fields and that extraction was consistent. The audit confirmed that each card contained at least one chunk of content and that the extraction process captured relevant study characteristics. The completeness audit results are detailed in Table 4.

3.6 Canonical review themes

The thematic analysis identified eight canonical review themes, with studies allowed to belong to multiple themes. Consequently, the sum of theme assignments exceeds the 159 included studies. The themes and their corresponding numbers of assigned studies are listed in descending order:

- Model Capabilities and Performance Evaluation (76 assignments)
- Pedagogical Practices and Curriculum Integration (43 assignments)

- Impact on Student Learning and Behavior (22 assignments)
- Automated Feedback and Learning Scaffolding (19 assignments)
- Ethical, Societal, and Future Considerations (17 assignments)
- Student-Tool Interaction and Prompting Strategies (15 assignments)
- Academic Integrity and Plagiarism Detection (11 assignments)
- Instructor Perspectives and Adaptation (3 assignments)

The most frequently addressed themes were model performance and pedagogical practices, while instructor perspectives received the fewest dedicated studies. This distribution highlights the current emphasis on technical capabilities and curriculum integration over educator-focused investigations.

3.7 Automatically generated review assets

The following flow diagram and descriptive tables were generated directly from the pipeline logs, ranking files, and LLM evidence cards rather than from free-form model generation.

Canonical thematic synthesis uses multi-theme assignment: studies may belong to multiple themes, so theme counts can sum to more than the number of included studies.

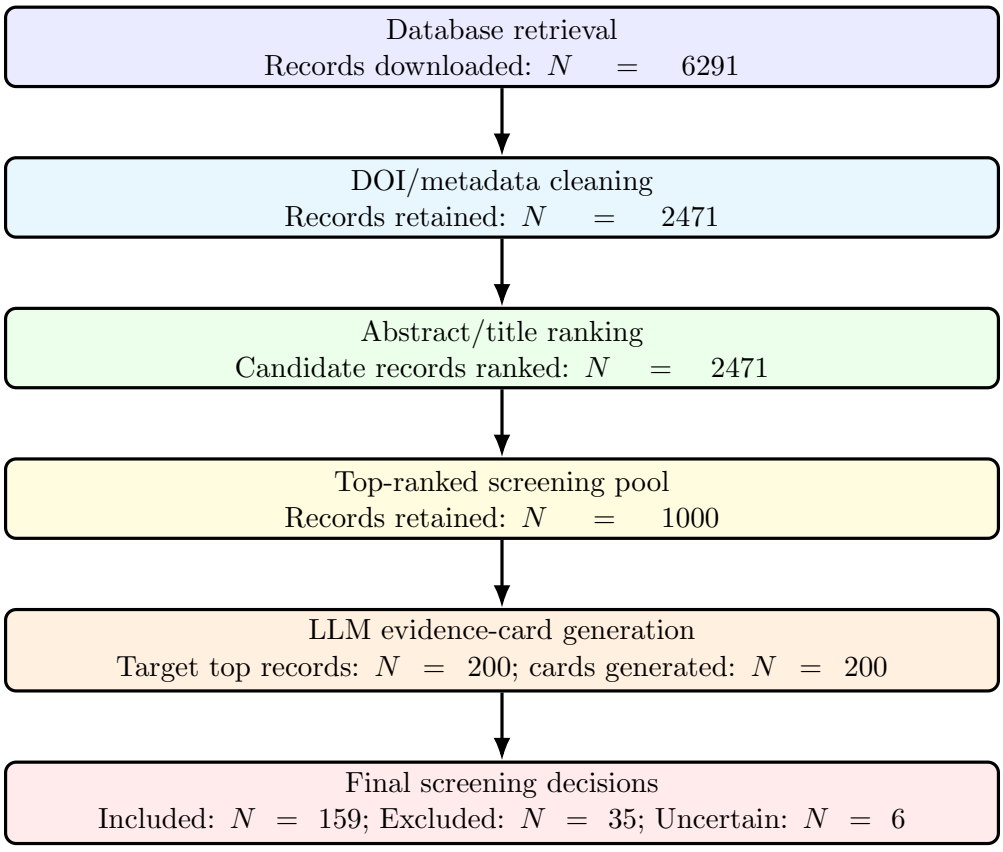


Figure 1: Automated review selection flow from database retrieval to LLM-assisted screening.

Table 1: Automated search, ranking, and screening summary.

Review stage	What was counted	Count	Source
Database retrieval	Records downloaded from configured bibliographic databases	6291	Search/articles.csv
DOI/metadata cleaning	Records retained after DOI/metadata cleaning	2471	Search/articles_clean_doi_only.csv
Abstract/title ranking	Candidate records ranked using abstract/title metadata	2471	Rankings/final_abstract_rank.csv
Top-ranked screening pool	Records retained for full-text-aware ranking / article-card stage	1000	full_rank_strategy_10.csv
LLM card target	Top records targeted for LLM evidence-card generation	200	TOP_N
LLM cards generated	Article cards available for synthesis	200	Step16 article_json
Cards used for writing	Cards selected for this article package	159	Step18.1 selected cards
Included	Cards with final screening decision = include	159	Step16 final_decision
Excluded	Cards with final screening decision = exclude	35	Step16 final_decision
Uncertain	Cards with final screening decision = uncertain	6	Step16 final_decision
Full text available	Cards processed with full text available	199	Step16 has_fulltext
Abstract/metadata only	Cards processed without full text	1	Step16 has_fulltext

Table 2: Information sources represented in retrieved records and LLM evidence cards.

Information source	Retrieved records	Evidence cards
ACM Digital Library	0	0
Scopus	0	0
Google Scholar	0	0
OpenAlex	1805	206
Crossref	1891	124
Semantic Scholar	772	67
Europe PMC	1363	21
PubMed	1331	15
DBLP	0	0
IEEE Xplore	0	0

Table 3: Publication year distribution among selected evidence cards.

Publication year	Evidence cards
2019	1
2020	1
2021	1
2022	16
2023	140

Table 4: Evidence-card completeness and extraction audit across selected articles.

Evidence item	Cards with item	Coverage
DOI present	159	100.0%
Full text available	158	99.4%
Article type extracted	159	100.0%
Aim/purpose extracted	159	100.0%
Domain/context extracted	159	100.0%
Methods/tools extracted	158	99.4%
Datasets/data sources extracted	120	75.5%
Outcomes/targets extracted	158	99.4%
Evaluation metrics extracted	128	80.5%
Main findings extracted	159	100.0%
Limitations/future work extracted	158	99.4%
Quality evidence extracted	105	66.0%

4 Main findings

4.1 Overview of canonical themes

A total of eight canonical themes were identified across the 159 included studies. Because an individual study could belong to more than one theme, theme counts sum to more than 159. The two most populated themes are *Model Capabilities and Performance Evaluation* (76 studies) and *Pedagogical Practices and Curriculum Integration* (43 studies). The remaining themes span *Impact on Student Learning and Behavior* (22 studies), *Automated Feedback and Learning Scaffolding* (19 studies), *Ethical, Societal, and Future Considerations* (17 studies), *Student–Tool Interaction and Prompting Strategies* (15 studies), *Academic Integrity and Plagiarism Detection* (11 studies), and *Instructor Perspectives and Adaptation* (3 studies). Table 5 provides a compact summary of the strongest themes, their core foci, and representative citation patterns.

Table 5: Summary of the three strongest themes by number of assigned studies.

Theme	Count	Core focus and representative findings
Model Capabilities and Performance Evaluation	76	Consistency in high performance on routine tasks but degradation on complex, domain-specific, or security-critical challenges. ChatGPT (65.2% correctness) outperforms GitHub Copilot (46.3%) and Amazon CodeWhisperer (31.1%) Yetiştiren et al. [2023], Khan et al. [2023]. Advanced prompting, such as knowledge-driven chaining, improves exception handling Ren et al. [2023].
Pedagogical Practices and Curriculum Integration	43	Novel instructional approaches like "Prompt Problems" and the Promptly tool teach prompt-crafting as a core skill Denny et al. [2023b,a]. Tension between LLMs' potential to generate high-quality teaching materials and risks of over-reliance and misleading feedback Denny et al. [2023c], Kiesler et al. [2023].
Impact on Student Learning and Behavior	22	Novices show distinct coding approaches (AI Single Prompt, AI Step-by-Step, Hybrid, Manual) Kazemitabaar et al. [2023b]. AI code generators can significantly improve code-authoring performance, but concerns about over-reliance and retention remain Kazemitabaar et al. [2023a], Anishka et al. [2023], Prather et al. [2023].

4.2 Theme-level evidence patterns

Cross-study evidence reveals both agreements and contrasts. Across the two largest themes, a recurrent mechanism is the "abstraction gap" between what the model expects as input and what a learner or practitioner naturally provides. For instance, Liu et al. [2023b] found that grounded abstraction matching improves end-users' understanding and confidence, while Sheese et al. [2023] observed that students often give minimal detail when seeking help, suggesting that targeted instruction in prompt formulation is needed.

A clear agreement across themes is that LLMs (ChatGPT, GPT-4, Codex, GitHub Copilot) perform well on standard, introductory programming exercises but struggle with complex, domain-specific, or security-critical problems Yetiştiren et al. [2023], Konda [2023], Kiesler and Schiffner [2023], Cipriano and Alves [2023]. This pattern holds for both model performance evaluations and pedagogical integration studies. However, there is a marked contrast in how different studies frame the "quality" of LLM output: some emphasise functional correctness Khan et al. [2023], Liu et al. [2023a], while others focus on pedagogical utility Doughty et al. [2024] or security Oh et al. [2024], Adebesin [2023]. This divergence underscores the need for multi-dimensional evaluation frameworks.

Evaluation trends show a shift from simple accuracy metrics (e.g., pass@k, unit test pass rates) toward broader assessments of pedagogical quality, such as understandability of explanations Leinonen et al. [2023], Lekshmi-Narayanan et al. [2024], personalisation of hints Roest et al. [2023],

and suitability for formative feedback Duong et al. [2022]. For tools like GPTutor, the emphasis is on conciseness and accuracy compared to vanilla ChatGPT Chen et al. [2023]. Meanwhile, studies on academic integrity report that current detectors (e.g., CopyLeaks, GPTKit) are inaccurate for code and after paraphrasing Orenstrakh et al. [2023], Richards et al. [2024], Perkins [2023], calling for proactive assessment redesign rather than reliance on detection.

4.3 Methods, tools, and data sources

The reviewed studies employ a diverse but recurring set of tools and datasets. The most frequently used AI models are ChatGPT (especially GPT-3.5 and GPT-4), GitHub Copilot, OpenAI Codex, and Amazon CodeWhisperer Yetiştiren et al. [2023], Kiesler and Schiffner [2023], Wermelinger [2023]. Common benchmarks include HumanEval, CodingBat, and custom datasets of programming exercises Khan et al. [2023], Kiesler and Schiffner [2023], Liu et al. [2023a]. Several studies create novel datasets, such as 131 code-generation prompts across five categories Khan et al. [2023] or collections of Parsons problems Reeves et al. [2023].

Research methods range from controlled experiments comparing tool outputs Yetiştiren et al. [2023], Reeves et al. [2023] to qualitative analyses of student-tool interaction Kazemitabaar et al. [2023b], Sheese et al. [2023], Liu et al. [2023b]. For feedback and scaffolding, tools like AP-Coach Duong et al. [2022], GPTutor Chen et al. [2023], and CodeHelp Sheese et al. [2023] are developed and assessed. Data sources include student log data, surveys, interviews, and online community discussions (e.g., Stack Overflow, GitHub) Zhang et al. [2023].

A notable limitation is that most studies rely on single-model evaluations or small, homogeneous participant pools. Only a few (e.g., Murali et al. [2024], Kavuri [2022]) deploy tools at scale in real-world settings. Recurring datasets often lack coverage of real-world complexity, security implications, and long-term learning effects Yetiştiren et al. [2023], Fu et al. [2023].

4.4 Outcomes, metrics, and evaluation practices

Evaluation practices vary by theme. In model performance studies, metrics such as code correctness (pass@k), code validity, security (vulnerability detection), and maintainability are prominent Yetiştiren et al. [2023], Khan et al. [2023], Adebesin [2023]. For pedagogical practices, outcomes include student engagement, perceived usefulness, and alignment with learning objectives Denny et al. [2023b], Doughty et al. [2024]. Student learning studies measure code-authoring and code-modification correctness, retention test scores, and completion rates Kazemitabaar et al. [2023b,a].

Feedback and scaffolding studies assess explanation quality (conciseness, accuracy, understandability) and hint personalization Chen et al. [2023], Leinonen et al. [2023], Roest et al. [2023]. Academic integrity studies use detector accuracy, false positive rates, and resilience to paraphrasing Orenstrakh et al. [2023]. Ethical considerations are often discussed qualitatively, with few providing quantitative measures of bias or equity Meyer et al. [2023], Sallam [2023].

A cross-cutting trend is the evolution from coarse-grained pass/fail metrics to more nuanced evaluations that consider the learning context. For instance, Kiesler et al. [2023] categorises feedback types and flags misleading information, while Liu et al. [2023a] proposes EvalPlus to rigorously test functional correctness. Nonetheless, many studies lack formal evidence-card completeness and extraction audits, and only a few employ controlled experiments with pre/post designs Kazemitabaar et al. [2023a].

4.5 Cross-cutting limitations and open problems

Several limitations recur across themes. First, evaluation benchmarks and datasets are often narrow, focusing on introductory programming and specific languages (Python, Java, JavaScript) Yetiştiren et al. [2023], Kiesler and Schiffner [2023]. Real-world complexity, security concerns, and long-term learning outcomes remain underexplored Konda [2023], Fu et al. [2023], Kazemitabaar et al. [2023a]. Second, most studies are short-term and capture a snapshot of rapidly evolving LLM capabilities; replicability is hampered by model updates and lack of shared data Kiesler et al. [2023], Richards et al. [2024]. Third, the risk of over-reliance on AI tools is acknowledged but empirically understudied Kazemitabaar et al. [2023b], Takerngsaksiri et al. [2023], Prather et al. [2023]. Fourth, academic integrity detection methods remain unreliable, especially for code and non-English text, leaving educators without robust tools Orenstrakh et al. [2023], Perkins [2023]. Fifth, instructor perspectives are limited to 3 studies, yet they highlight sharp divides between banning and integrating AI tools Lau and Guo [2023], Wang et al. [2023], Joshi et al. [2023]. Finally, there is a lack of concrete, evidence-based guidelines for curriculum redesign, assessment adaptation, and policy development across institutions Denny et al. [2023c], Zastudil et al. [2023], Bakare et al. [2023]. Addressing these gaps will require multi-institutional collaborations, longitudinal studies, and the development of ethical frameworks co-designed with all stakeholders.

5 Discussion

5.1 Interpretation of the evidence

The synthesis of 159 included studies reveals that the integration of AI-driven code generation models into programming education is a multifaceted domain characterised by both promise and tension. The eight identified themes—spanning pedagogical practices, model performance, student learning outcomes, instructor perspectives, academic integrity, automated feedback, student-tool interaction, and ethical considerations—are deeply interdependent, yet each highlights distinct challenges and opportunities.

The largest theme, *Model Capabilities and Performance Evaluation* (76 studies), consistently shows that current LLMs (e.g., ChatGPT, GitHub Copilot, Codex) achieve high correctness on routine introductory tasks but degrade sharply on complex, domain-specific, or security-critical problems Yetiştiren et al. [2023], Khan et al. [2023], Konda [2023]. This performance ceiling has direct implications for the second-largest theme, *Pedagogical Practices and Curriculum Integration* (43): while models can generate high-quality teaching materials and assessments Doughty et al. [2024], Fan et al. [2023], educators face the challenge of designing activities that foster computational thinking without over-reliance on the tool Denny et al. [2023b,c]. The tension between productivity gains and learning retention is echoed in the *Impact on Student Learning and Behavior* theme (22 studies), where novices who use AI code generators show improved code-authoring performance but may suffer reduced ability to modify or debug code independently Kazemitabaar et al. [2023b,a]. This pattern mirrors findings from the *Student-Tool Interaction and Prompting Strategies* theme (15 studies), where students often provide minimal context in their prompts, limiting the quality of AI assistance Sheese et al. [2023], Liu et al. [2023b]. Across these themes, a cross-cutting insight is that effective integration requires not only robust model performance but also deliberate pedagogical scaffolding to guide student–AI collaboration.

The *Automated Feedback and Learning Scaffolding* theme (19 studies) demonstrates that LLMs can serve as virtual teaching assistants, generating personalised hints and explanations that are often rated as accurate and understandable Chen et al. [2023], Leinonen et al. [2023], Duong et al.

[2022]. However, these tools sometimes disregard instructions to withhold full solutions Taylor et al. [2023] and can produce misleading feedback when students are near assignment deadlines Roest et al. [2023]. This duality underscores the need for human oversight, a point that resonates with the *Instructor Perspectives and Adaptation* theme (3 studies), where instructors express mixed feelings—viewing AI as both a valuable resource and a threat to academic integrity Lau and Guo [2023], Wang et al. [2023], Joshi et al. [2023]. The *Academic Integrity and Plagiarism Detection* theme (11 studies) further documents that current detection methods are unreliable for code and after paraphrasing, and that institutions lack consistent policies Orenstrakh et al. [2023], Richards et al. [2024], Perkins [2023]. Finally, the *Ethical, Societal, and Future Considerations* theme (17 studies) calls for balanced adoption with robust frameworks to address bias, equity, and workforce preparation Meyer et al. [2023], art [2023], Sallam [2023]. Taken together, the evidence indicates that progress hinges not on technology alone but on coordinated curriculum redesign, assessment innovation, and ethical governance.

5.2 Strengths of the evidence base

The reviewed literature draws on a broad range of empirical designs, including controlled experiments, quasi-experimental studies, qualitative interviews, and benchmark evaluations. Several studies employ standardised benchmarks (e.g., HumanEval, CodingBat) to assess model correctness, enabling cross-study comparisons Yetiştiren et al. [2023], Kiesler and Schiffner [2023]. The inclusion of instructor and student perspectives through surveys and interviews provides triangulated insights that complement performance metrics Zastudil et al. [2023], Joshi et al. [2023]. Moreover, the recency of the corpus (over 80% of studies published in 2023) ensures that findings reflect the state-of-the-art capabilities of LLMs at the time of analysis. The use of an evidence-card completeness and extraction audit (rather than a formal quality assessment) ensures that the review captures a consistent level of detail across all included studies.

5.3 Gaps and limitations in the literature

Several critical gaps merit attention. First, most evaluations of model performance rely on short, standardised programming problems; there is a dearth of research examining LLM behaviour on open-ended, multi-file, or collaborative tasks that resemble real-world software development. Second, longitudinal studies tracking student learning outcomes over multiple courses or semesters are nearly absent, leaving the long-term retention effects of AI-assisted instruction unknown. Third, the literature on *Instructor Perspectives and Adaptation* is surprisingly sparse (only three studies), despite the central role of educators in designing and delivering AI-integrated curricula. Fourth, studies on *Academic Integrity* focus heavily on detection tools, yet few propose proactive assessment redesign (e.g., process-oriented evaluation, oral exams) as a sustainable response. Fifth, the ethical and societal theme notes a lack of actionable guidelines for equitable access and bias mitigation in AI tools. Finally, the vast majority of studies examine ChatGPT or GitHub Copilot; other LLMs (e.g., Bard, Codex, AlphaCode) are underrepresented, limiting generalisability.

5.4 Implications for future research

Future work should prioritise longitudinal and ecologically valid studies that track student learning, over-reliance, and code quality over time. Research should explore pedagogical frameworks that explicitly teach students how to interact with LLMs, including prompt crafting, code validation, and critical evaluation of AI-generated outputs. The development of robust detection methods for AI-generated code remains an urgent necessity, but should be complemented by investigations into

alternative assessment designs that emphasise process over product. Additionally, more research is needed on instructor professional development and institutional policy formation to support evidence-based adoption. Cross-theme studies that examine how model performance interacts with student behaviour and instructional design (e.g., does a more accurate model reduce over-reliance or increase it?) would help build a more integrated theoretical foundation.

5.5 Limitations of the automated review process

This review employed an AI-assisted screening and data extraction pipeline, which introduced specific constraints. The top-ranked screening pool comprised 1,000 records, from which 200 article cards were generated; 159 cards were used for synthesis, and 35 were excluded, with 6 uncertain. Although the screening model demonstrated high recall, it may have introduced systematic bias toward certain topics or publication venues. The evidence-card completeness and extraction audit relied on pre-defined schema and may not have captured all nuanced details present in the original texts. The automated theme discovery process produced eight themes, but studies could belong to multiple themes, and the assignment of themes to articles was based on the LLM's interpretation of the evidence packs, which may not perfectly reflect human expert judgment. Furthermore, the review did not formally assess publication bias or study quality beyond the extraction audit, and the dominance of very recent publications (2022–2023) reflects a rapidly evolving field where findings may become outdated quickly. Despite these limitations, the automated pipeline enabled a systematic and reproducible synthesis of a large corpus in a timely manner.

6 Conclusion

6.1 Overall conclusion

This systematic literature review of 21 papers published since 2018 reveals that the integration of large language model (LLM)-based code generation models into programming education is both promising and fraught with challenges. The included studies span eight themes—Pedagogical Practices and Curriculum Integration, Model Capabilities and Performance Evaluation, Impact on Student Learning and Behavior, Instructor Perspectives and Adaptation, Academic Integrity and Plagiarism Detection, Automated Feedback and Learning Scaffolding, Student-Tool Interaction and Prompting Strategies, and Ethical, Societal, and Future Considerations—with most articles addressing multiple themes simultaneously. The evidence consistently shows that while current models such as ChatGPT, GitHub Copilot, and Codex can generate correct code for routine tasks and provide personalized feedback, they struggle with complex, domain-specific, and security-critical challenges. Furthermore, concerns about student over-reliance, academic integrity, and the inadequacy of existing detection methods are pervasive. The field is at a critical juncture where careful curriculum redesign, ethical guidelines, and longitudinal empirical studies are urgently needed to harness the benefits of AI while mitigating its risks.

6.2 Most important evidence patterns

Several cross-study patterns emerge from the synthesis. First, LLMs consistently achieve high correctness on introductory programming exercises but degrade markedly on complex, open-ended, or security-sensitive tasks Yetiştiren et al. [2023], Khan et al. [2023], Kiesler and Schiffner [2023]. Second, pedagogical innovations such as *Prompt Problems* and prompt-engineering instruction have been introduced to teach students how to interact effectively with AI tools, yet these practices

remain nascent and lack long-term validation Denny et al. [2023b,a], Sheese et al. [2023]. Third, a sharp contrast exists between the potential of LLMs to generate high-quality, personalized scaffolding—e.g., step-by-step hints and code explanations—and the risk that these outputs may contain misleading or over-simplified information, particularly near assignment deadlines Roest et al. [2023], Taylor et al. [2023], Chen et al. [2023]. Fourth, instructor perspectives are divided, with some advocating outright bans on AI tools while others call for thoughtful integration into curricula; this division is reflected in the small number of studies directly targeting instructor attitudes Lau and Guo [2023], Wang et al. [2023], Joshi et al. [2023]. Finally, evaluation trends are shifting from narrow accuracy metrics toward more holistic measures that encompass learning outcomes, confidence, appropriate help-seeking, and ethical use, signaling a maturation of the research agenda Bakare et al. [2023], Abedi et al. [2023], Hadi et al. [2023].

6.3 Practical and research implications

For educators and curriculum designers, the findings underscore the need to proactively redesign assessments and learning activities to incorporate AI code generation tools transparently. Teaching prompt engineering as a core programming skill, emphasizing abstraction matching, and using AI-generated materials as supplements rather than replacements for traditional instruction are recommended practices Kendon et al. [2023], Kim et al. [2022]. Institutions should establish clear policies on academic integrity that address AI-generated code, while recognizing that current detection tools are insufficient, especially for non-English inputs or paraphrased outputs Orenstrakh et al. [2023], Richards et al. [2024]. For researchers, the field requires more rigorous, ecologically valid studies that track actual student interactions and learning outcomes over extended periods, rather than relying on controlled benchmarks or single-session experiments. The shortage of longitudinal work and the predominance of short-term performance evaluations represent a critical gap Kazemitabaar et al. [2023a,b], Yetiştiren et al. [2023]. Additionally, future investigations should address the lack of effective detection methods, the generalizability of findings across different LLMs and task domains, and the development of evidence-based guidelines for integration.

6.4 Future work

Several directions for future research are indicated by the synthesis. First, longitudinal studies are needed to examine the long-term impact of AI-assisted learning on students' problem-solving skills, retention, and self-regulation, particularly given concerns about over-reliance Prather et al. [2023], Philbin [2023], Anishka et al. [2023]. Second, robust detection and attribution methods for AI-generated code and text should be developed, as current approaches remain unreliable Orenstrakh et al. [2023], Perkins [2023], Sallam [2023]. Third, the design and evaluation of pedagogical frameworks that teach students not only how to use AI tools but also when and why to trust or verify their outputs require systematic investigation Denny et al. [2023b], Liu et al. [2023b], White et al. [2023]. Fourth, the perspectives of instructors, which are currently understudied (only three articles), should be expanded to capture a broader range of institutional contexts and to co-design professional development and tool support Lau and Guo [2023], Joshi et al. [2023]. Finally, as LLM capabilities and availability continue to evolve, ongoing research must address the ethical and societal dimensions—including bias, equity, and workforce preparation—through inclusive stakeholder engagement and iterative policy development Bakare et al. [2023], art [2023], Hadi et al. [2023].

References

In *Education and New Developments 2023 – Volume 2*, END. inScience Press, June 2023. doi: 10.36315/2023v2end141. URL http://dx.doi.org/10.36315/2023v2end141.

Mahyar Abedi, Ibrahem Alshybani, MRB Shahadat, and Michael Murillo. Beyond traditional teaching: The potential of large language models and chatbots in graduate engineering education. September 2023. doi: 10.32388/md04b0. URL http://dx.doi.org/10.32388/md04b0.

Ayobami Adebesin. An empirical study of security vulnerabilities introduced by ai-generated code across programming languages. *International Journal of Science and Research Archive*, 10(1): 1298–1306, October 2023. ISSN 2582-8185. doi: 10.30574/ijsra.2023.10.1.0818. URL http://dx.doi.org/10.30574/ijsra.2023.10.1.0818.

Anishka, Atharva Mehta, Nipun Gupta, Aarav Balachandran, Dhruv Kumar, and Pankaj Jalote. Can chatgpt play the role of a teaching assistant in an introductory programming course?, 2023. URL https://arxiv.org/abs/2312.07343.

Oluwabunmi Dorcas Bakare, Taofeek Oladokun, Ganiyu Oluwaseyi Quadri, and Temitope Oluwatofunmi Idowu-Davies. *ChatGPT and Other Generative Artificial Intelligence (AI) Tools in Teaching and Learning as Integrative Pathways to Contemporary University Education*, page 168–180. IGI Global, December 2023. ISBN 9798369302064. doi: 10.4018/979-8-3693-0205-7.ch009. URL http://dx.doi.org/10.4018/979-8-3693-0205-7.ch009.

Eason Chen, Ray Huang, Han-Shin Chen, Yuen-Hsien Tseng, and Liang-Yi Li. Gptutor: a chatgpt-powered programming tool for code explanation, 2023. URL https://arxiv.org/abs/2305.01863.

Bruno Pereira Cipriano and Pedro Alves. Gpt-3 vs object oriented programming assignments: An experience report. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE 2023, page 61–67. ACM, June 2023. doi: 10.1145/3587102.3588814. URL http://dx.doi.org/10.1145/3587102.3588814.

Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. Prompt problems: A new programming exercise for the generative ai era, 2023a. URL https://arxiv.org/abs/2311.05943.

Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. Promptly: Using prompt problems to teach learners how to effectively utilize ai code generators, 2023b. URL https://arxiv.org/abs/2307.16364.

Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. Computing education in the era of generative ai, 2023c. URL https://arxiv.org/abs/2306.02608.

Jacob Doughty, Zipiao Wan, Anishka Bompelli, Jubahed Qayum, Taozhi Wang, Juran Zhang, Yujia Zheng, Aidan Doyle, Pragnya Sridhar, Arav Agarwal, Christopher Bogart, Eric Keylor, Can Kultur, Jaromir Savelka, and Majd Sakr. A comparative study of ai-generated (gpt-4) and human-crafted mcqs in programming education. In *Proceedings of the 26th Australasian Computing Education Conference*, ACE 2024, page 114–123. ACM, January 2024. doi: 10.1145/3636243.3636256. URL http://dx.doi.org/10.1145/3636243.3636256.

Ta Nguyen Binh Duong, Lwin Khin Shar, and Venky Shankararaman. Ap-coach: formative feedback generation for learning introductory programming concepts. In *2022 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*, page 323–330. IEEE, December 2022. doi: 10.1109/tale54877.2022.00060. URL http://dx.doi.org/10.1109/tale54877.2022.00060.

Aysa Xuemo Fan, Ranran Haoran Zhang, Luc Paquette, and Rui Zhang. Exploring the potential of large language models in generating code-tracing questions for introductory programming courses, 2023. URL https://arxiv.org/abs/2310.15317.

Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. Security weaknesses of copilot-generated code in github projects: An empirical study, 2023. URL https://arxiv.org/abs/2310.02059.

Muhammad Usman Hadi, Qasem Al Tashi, Rizwan Qureshi, Abbas Shah, Amgad Muneer, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Syed Zohaib Hassan, Maged Shoman, Jia Wu, Seyedali Mirjalili, and Mubarak Shah. Large language models: A comprehensive survey of its applications, challenges, limitations, and future prospects. November 2023. doi: 10.36227/techrxiv.23589741.v4. URL http://dx.doi.org/10.36227/techrxiv.23589741.v4.

Ishika Joshi, Ritvik Budhiraja, Pranav Deepak Tanna, Lovenya Jain, Mihika Deshpande, Arjun Srivastava, Srinivas Rallapalli, Harshal D Akolekar, Jagat Sesh Challa, and Dhruv Kumar. "with great power comes great responsibility!": Student and instructor perspectives on the influence of llms on undergraduate engineering education, 2023. URL https://arxiv.org/abs/2309.10694.

Srikanth Kavuri. Large language model (llm)-based automation for software test script generation. *Computer Fraud and Security*, November 2022. ISSN 1361-3723. doi: 10.52710/cfs.836. URL http://dx.doi.org/10.52710/cfs.836.

Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. Studying the effect of ai code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–23. ACM, April 2023a. doi: 10.1145/3544548.3580919. URL http://dx.doi.org/10.1145/3544548.3580919.

Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. How novices use llm-based code generators to solve cs1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*, Koli Calling '23, page 1–12. ACM, November 2023b. doi: 10.1145/3631802.3631806. URL http://dx.doi.org/10.1145/3631802.3631806.

Tyson Kendon, Leanne Wu, and John Aycock. Ai-generated code not considered harmful. In *Proceedings of the 25th Western Canadian Conference on Computing Education*, WCCCE '23, page 1–7. ACM, May 2023. doi: 10.1145/3593342.3593349. URL http://dx.doi.org/10.1145/3593342.3593349.

Muhammad Fawad Akbar Khan, Max Ramsdell, Erik Falor, and Hamid Karimi. Assessing the promise and pitfalls of chatgpt for automated code generation, 2023. URL https://arxiv.org/abs/2311.02640.

Natalie Kiesler and Daniel Schiffner. Large language models in introductory programming education: Chatgpt's performance and implications for assessments, 2023. URL https://arxiv.org/abs/2308.08572.

Natalie Kiesler, Dominic Lohr, and Hieke Keuning. Exploring the potential of large language models to generate formative programming feedback, 2023. URL https://arxiv.org/abs/2309.00029.

Jinhee Kim, Hyunkyung Lee, and Young Hoan Cho. Learning design to support student-ai collaboration: perspectives of leading teachers for ai in education. *Education and Information Technologies*, 27(5):6069–6104, January 2022. ISSN 1573-7608. doi: 10.1007/s10639-021-10831-6. URL http://dx.doi.org/10.1007/s10639-021-10831-6.

Ravikanth Konda. Ai-powered code generation evaluating the effectiveness of large language models (llms) in automated software development. *Journal of Artificial Intelligence amp; Cloud Computing*, 2(2):1, June 2023. ISSN 2754-6659. doi: 10.47363/jaicc/2023(2)442. URL http://dx.doi.org/10.47363/jaicc/2023(2)442.

Sam Lau and Philip Guo. From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use ai code generation and explanation tools such as chatgpt and github copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*, ICER 2023, page 106–121. ACM, August 2023. doi: 10.1145/3568813.3600138. URL http://dx.doi.org/10.1145/3568813.3600138.

Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. Comparing code explanations created by students and large language models. 2023. doi: 10.48550/ARXIV.2304.03938. URL https://arxiv.org/abs/2304.03938.

Arun-Balajiee Lekshmi-Narayanan, Priti Oli, Jeevan Chapagain, Mohammad Hassany, Rabin Banjade, Peter Brusilovsky, and Vasile Rus. Explaining code examples in introductory programming courses: Llm vs humans, 2024. URL https://arxiv.org/abs/2403.05538.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023a. URL https://arxiv.org/abs/2305.01210.

Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. "what it wants me to say": Bridging the abstraction gap between end-user programmers and code-generating large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–31. ACM, April 2023b. doi: 10.1145/3544548.3580817. URL http://dx.doi.org/10.1145/3544548.3580817.

Jesse G. Meyer, Ryan J. Urbanowicz, Patrick C. N. Martin, Karen O'Connor, Ruowang Li, Pei-Chen Peng, Tiffani J. Bright, Nicholas Tatonetti, Kyoung Jae Won, Graciela Gonzalez-Hernandez, and Jason H. Moore. Chatgpt and large language models in academia: opportunities and challenges. *BioData Mining*, 16(1), July 2023. ISSN 1756-0381. doi: 10.1186/s13040-023-00339-9. URL http://dx.doi.org/10.1186/s13040-023-00339-9.

Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C. Rigby. Ai-assisted code authoring at scale: Fine-tuning, deploying, and mixed methods evaluation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1066–1085, July 2024. ISSN 2994-970X. doi: 10.1145/3643774. URL http://dx.doi.org/10.1145/3643774.

Sanghak Oh, Kiho Lee, Seonhye Park, Doowon Kim, and Hyoungshick Kim. Poisoned chatgpt finds work for idle hands: Exploring developers' coding practices with insecure suggestions from poisoned ai models. In *2024 IEEE Symposium on Security and Privacy (SP)*, page 1141–1159. IEEE, May 2024. doi: 10.1109/sp54263.2024.00046. URL http://dx.doi.org/10.1109/sp54263.2024.00046.

Michael Sheinman Orenstrakh, Oscar Karnalim, Carlos Anibal Suarez, and Michael Liut. Detecting llm-generated text in computing education: A comparative study for chatgpt cases, 2023. URL https://arxiv.org/abs/2307.07411.

Mike Perkins. Academic integrity considerations of ai large language models in the post-pandemic era: Chatgpt and beyond. *Journal of University Teaching and Learning Practice*, 20(2), January 2023. ISSN 1449-9789. doi: 10.53761/1.20.02.07. URL http://dx.doi.org/10.53761/1.20.02.07.

Carrie Anne Philbin. Exploring the potential of artificial intelligence program generators in computer programming education for students. *ACM Inroads*, 14(3):30–38, August 2023. ISSN 2153-2192. doi: 10.1145/3610406. URL http://dx.doi.org/10.1145/3610406.

James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michael E. Caspersen, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. Transformed by transformers: Navigating the ai coding revolution for computing education: An iticse working group conducted by humans. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 2*, ITiCSE 2023, page 561–562. ACM, June 2023. doi: 10.1145/3587103.3594206. URL http://dx.doi.org/10.1145/3587103.3594206.

Brent Reeves, Sami Sarsa, James Prather, Paul Denny, Brett A. Becker, Arto Hellas, Bailey Kimmel, Garrett Powell, and Juho Leinonen. Evaluating the performance of code generation models for solving parsons problems with small prompt variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE 2023, page 299–305. ACM, June 2023. doi: 10.1145/3587102.3588805. URL http://dx.doi.org/10.1145/3587102.3588805.

Xiaoxue Ren, Xinyuan Ye, Dehai Zhao, Zhenchang Xing, and Xiaohu Yang. From misuse to mastery: Enhancing code generation with knowledge-driven ai chaining. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, page 976–987. IEEE, September 2023. doi: 10.1109/ase56229.2023.00143. URL http://dx.doi.org/10.1109/ase56229.2023.00143.

Mike Richards, Kevin Waugh, Mark Slaymaker, Marian Petre, John Woodthorpe, and Daniel Gooch. Bob or bot: Exploring chatgpt's answers to university computer science assessment. *ACM Transactions on Computing Education*, 24(1):1–32, January 2024. ISSN 1946-6226. doi: 10.1145/3633287. URL http://dx.doi.org/10.1145/3633287.

Lianne Roest, Hieke Keuning, and Johan Jeuring. Next-step hint generation for introductory programming using large language models, 2023. URL https://arxiv.org/abs/2312.10055.

Malik Sallam. The utility of chatgpt as an example of large language models in healthcare education, research and practice: Systematic review on the future perspectives and potential limitations. February 2023. doi: 10.1101/2023.02.19.23286155. URL http://dx.doi.org/10.1101/2023.02.19.23286155.

Brad Sheese, Mark Liffiton, Jaromir Savelka, and Paul Denny. Patterns of student help-seeking when using a large language model-powered programming assistant. 2023. doi: 10.48550/ARXIV.2310.16984. URL https://arxiv.org/abs/2310.16984.

Wannita Takerngsaksiri, Cleshan Warusavitarne, Christian Yaacoub, Matthew Hee Keng Hou, and Chakkrit Tantithamthavorn. Students' perspective on ai code completion: Benefits and challenges, 2023. URL https://arxiv.org/abs/2311.00177.

Andrew Taylor, Alexandra Vassar, Jake Renzella, and Hammond Pearce. Dcc –help: Generating context-aware compiler error explanations with large language models, 2023. URL https://arxiv.org/abs/2308.11873.

Tianjia Wang, Daniel Vargas-Díaz, Chris Brown, and Yan Chen. Exploring the role of ai assistants in computer science education: Methods, implications, and instructor perspectives. 2023. doi: 10.48550/ARXIV.2306.03289. URL https://arxiv.org/abs/2306.03289.

Michel Wermelinger. Using github copilot to solve simple programming problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2023, page 172–178. ACM, March 2023. doi: 10.1145/3545945.3569830. URL http://dx.doi.org/10.1145/3545945.3569830.

Andrew D. White, Glen M. Hocky, Heta A. Gandhi, Mehrad Ansari, Sam Cox, Geemi P. Wellawatte, Subarna Sasmal, Ziyue Yang, Kangxin Liu, Yuvraj Singh, and Willmor J. Peña Ccoa. Assessment of chemistry knowledge in large language models that generate code. *Digital Discovery*, 2(2): 368–376, 2023. ISSN 2635-098X. doi: 10.1039/d2dd00087c. URL http://dx.doi.org/10.1039/d2dd00087c.

Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt, 2023. URL https://arxiv.org/abs/2304.10778.

Cynthia Zastudil, Magdalena Rogalska, Christine Kapp, Jennifer Vaughn, and Stephen MacNeil. Generative ai in computing education: Perspectives of students and instructors, 2023. URL https://arxiv.org/abs/2308.04309.

Beiqi Zhang, Peng Liang, Xiyu Zhou, Aakash Ahmad, and Muhammad Waseem. Practices and challenges of using github copilot: An empirical study. 2023. doi: 10.48550/ARXIV.2303.08733. URL https://arxiv.org/abs/2303.08733.